

A – 三座城堡

Page 1 of 2

内存限制: 1024 MB 时间限制: 4 秒

EUC 2026
8.02.2026

在遥远的ICPC王国里，有 n 座塔。对于每座塔，我们都知道它在王国的位置 (x, y) 。为了方便起见，我们将每个塔视为地图上的一个点。保证没有两座塔位置相同，并且没有三座塔位于同一条线上。

EUC国王决定是时候根据以下规则建造三座城堡了：

- 每个城堡都包含一组非空的塔楼。
- 任何不属于任何城堡的塔都不应被孤立。
- 每座城堡都应该在王国地图上形成一个凸多边形，其所有塔楼都位于该多边形的顶点。我们假设拥有一个塔楼的城堡也是有效的。
- 任何一对城堡都不应该有共同点，无论是在边界还是在内部。

EUC国王尚未决定其三座城堡的塔楼数量。因此，他要求你

确定每个正整数三元组 s_1, s_2, s_3 （使得 $s_1 + s_2 + s_3 = n$ ），如果存在一个解，其中第 i 个城堡中的塔楼数量为 s_i （ $1 \leq i \leq 3$ ），如果存在这样的解决方案，请显示一个示例三座城堡之间的塔楼分布。

输入

输入的第一行包含一个整数 n （ $3 \leq n \leq 40$ ），表示塔的数量。塔是编号从1到 n 。

接下来的每一行 n 包含一座塔的坐标、两个整数 x 和 y （ $-10^6 \leq x, y \leq 10^6$ ）。正如已经提到的，没有两座塔处于同一位置，也没有三座塔位于同一位置内。

输出

输出的第一行应该包含数字 k ，即解决方案的不同三元组的数量存在。

接下来的每一行 k 都应包含三元组之一的解决方案。每行应包含3个 + n 个空格分隔的整数： $s_1, s_2, s_3, a_{1,1}, \dots, a_{1,s_1}, a_{2,1}, \dots, a_{2,s_2}, a_{3,1}, \dots, a_{3,s_3}$ ，其中 $a_{i,j}$ 表示他是第 i 个城堡中第 j 个塔楼的编号。

每个具有解决方案的无序三元组应该恰好出现一次，并且每个塔中的塔的顺序可以是任意的。

A – Three Castles

Page 2 of 2

Memory limit: 1024 MB

Time limit: 4 s

EUC 2026

8.02.2026

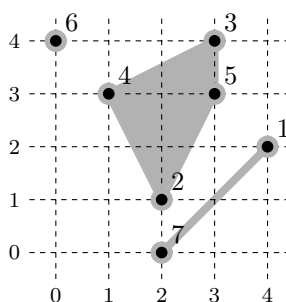
例子

对于输入数据:

```
7
4 2
2 1
3 4
1 3
3 3
0 4
2 0
```

一种可能的正确结果是:

```
3
1 3 3 7 6 3 4 2 5 1
4 2 1 3 4 5 2 7 1 6
2 2 3 4 5 2 7 1 6 3
```



解释: 有四个可能的三元组, 总和为 7。对于三元组 1, 2, 4, 一种解决方案是取城堡 {6}、{1, 7} 和 {2, 3, 4, 5}。三元组 1, 3, 3 和 2, 2, 3 也有解。三元组 1, 1, 5 没有解。请注意, {4}、{2, 5} 和 {1, 3, 6, 7} 不是三元组 1, 2, 4 的有效解, 因为第三座城堡与第一座和第二座城堡相交。请注意, {2}、{7} 和 {1, 3, 4, 5, 6} 不是三元组 1, 1, 5 的有效解, 因为第三座城堡不是凸的。

B – Bitwise Beach

Page 1 of 1

Memory limit: 1024 MB

Time limit: 1 s

EUC 2026

8.02.2026

每个朋友在海滩收集了 n^2 贝壳，并将它们排列成 n 行和 n 列的网格。他们然后在沙滩上划了一些水平线和一些垂直线，将海滩划分为多个区域，具有积极的效果 n 每个区域收集的贝壳数量。

现在，朋友们正在玩一款著名的 Nim 游戏。他们交替进行动作；在每一步中，玩家采取一来自任何单个区域的炮弹数量为正。拿下最后一颗炮弹的玩家获胜。

小伙伴们都知道，判断先手是否有必胜策略，以及该走哪一步棋。理想情况下，应该计算每个区域中的炮弹数量并计算这些数字的按位异或。时间起来工作量很大。帮助他们！

输入

输入的第一行包含一个整数 n ($1 \leq n \leq 10^6$)。我们假设行的编号为 1 到 n ，从上到下。这些列也从左到右编号为 1 到 n 。

第二行包含一串 $n - 1$ 位数字 0 或 1；如果存在水平线，则第 i 位 ($1 \leq i < n$) 为 1 行 i 和行 $i + 1$ 之间。

第三行包含一串 $n - 1$ 位数字 0 或 1；如果有垂直线，则第 i 位 ($1 \leq i < n$) 为 1 列 i 和列 $i + 1$ 之间 1。

输出

输出一个整数，即所有区域中壳数的按位异或。

例子

对于输入数据：

```
4
001
101
```

正确的结果是：

```
4
```

解释：共有 6 个区域，分别有 3、1、6、2、3、1 个壳。按位异或等于 $3 \oplus 1 \oplus 6 \oplus 2 \oplus 3 \oplus 1 = 4$ 。

○	○	○	○
○	○	○	○
○	○	○	○
○	○	○	○

C – Copy-paste

Page 1 of 1

Memory limit: 1024 MB

Time limit: 1 s

EUC 2026

8.02.2026

John 非常重视安全性，因此他的密码很长：有 n 个字符。然而，他并不是很敏感，因此密码中的所有字符都是相同的 - 字母 a 。John 很不耐烦，所以他想尽快输入密码。

他想要获得一个恰好由 n 个字母 a 组成的字符串。他可以使用以下一系列操作：“a”（输入字母 a ）、“Ctrl-A”（选择所有内容）、“Ctrl-C”（复制到剪贴板）、“Ctrl-V”（从剪贴板粘贴）。从空密码框和空剪贴板开始时，要达到所需结果，最少需要执行多少操作？

以下是 John 在密码框中可以执行的所有四种操作的精确定义：

- 按“a”：如果没有选择文本，则在光标位置（左侧）插入字母 a 。否则，将所选文本替换为字母 a 。之后，不会选择任何文本，并且光标位于密码框的末尾。
- 按“Ctrl-A”：选择密码框中的所有文本。
- 按“Ctrl-C”：如果未选择任何文本，则不执行任何操作。否则，将选定的文本复制到剪贴板。
- 按“Ctrl-V”：如果剪贴板为空，则不执行任何操作。否则，如果未选择任何文本，则将剪贴板中的文本插入到光标位置（左侧）。否则，将所选文本替换为剪贴板中的文本。最后，没有选择任何文本，并且光标位于密码框的末尾。剪贴板的内容不会改变。

输入

输入的第一行也是唯一一行包含一个整数 n ($1 \leq n \leq 10^{12}$)，表示所需的密码长度。

输出

输出一个整数，即输入所需长度的密码所需的最少操作次数。

例子

对于输入数据：11

正确的结果是：

10

乙解释：John 可以使用以下操作序列：a、a、Ctrl-A、Ctrl-C、Ctrl-V、Ctrl-V、Ctrl-V、Ctrl-V、Ctrl-V、a。

D – Deque Sort

Page 1 of 1

Memory limit: 1024 MB

Time limit: 4 s

EUC 2026
8.02.2026

时间是刚刚到达仓库的 n 个包裹，编号从 1 到 n 。他们以 $a_1, \dots, a_n$ 的顺序出现，
— 我们要对它们进行排序，使它们的顺序为 $1, 2, \dots, n$ 。

仓库内的分拣机分阶段工作。每个阶段接受一行 $a_1, \dots, a_n$ 地块，并按如下方式进行：首先，将第一个地块 a_1 放入新行中。然后，对于 $i = 2, \dots, n$ 的每个地块 a_i ，如果 a_i 大于新行中的最后一个（最右边）地块，则将其放在该行的末尾。否则，它会将其放在该行的前面。

如果新行未排序，则它将作为排序机下一阶段的输入。

编写一个程序，通过回答以下形式的查询来帮助测试机器是否按预期工作
“哪个地块位于 t 阶段之后的第 b 个位置”。

输入

输入的第一行包含两个整数 n 和 q ($1 \leq n, q \leq 5 \cdot 10^5$)，表示地块数量
 t 分别是查询的数量。

第二行包含一系列 n 整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n, a_i \neq a_j$) 表示包裹的初始订购。

接下来的 q 行包含查询；第 i 行包含两个整数 t_i 和 b_i ($0 \leq t_i \leq 10^9, 1 \leq b_i \leq n$)，
 d 表示第 i 个查询。

输出

准确输出 q 行。第 i 行应包含第 i 查询的答案。

如果机器执行的阶段少于 t_i 阶段，则写入 -1 。否则，写出哪个地块位于位置 b_i
执行完 t_i 阶段后，从左侧离开。

例子

对于输入数据：

```
5 4
2 1 3 5 4
1 1
3 2
4 1
0 4
```

正确的结果是：

```
4
2
-1
5
```

说明：机器将执行三个阶段：

2 1 3 5 4 → 4 1 2 3 5 → 3 2 1 4 5 → 1 2 3 4 5.

而对于输入数据：

```
1 1
1
1 1
```

正确的结果是：-1

E——评估

Page 1 of 1

内存限制：1024 MB 时间限制：1 秒

EUC 2026
8.02.2026

we 有一个由 n 个非零数字组成的字符串。我们可以通过插入最多一个来生成一个表达式
c任意两个连续数字之间的字符 $+$ (plus) 或 $\cdot$ (multiply)。例如，从字符串 “231” 我们
can 产生九个表达式：

$$231, \quad 23 + 1, \quad 23 \cdot 1, \quad 2 + 31, \quad 2 \cdot 31, \quad 2 + 3 + 1, \quad 2 + 3 \cdot 1, \quad 2 \cdot 3 + 1, \quad 2 \cdot 3 \cdot 1.$$

we 使用标准运算顺序计算这些表达式，并计算所有值的总和。为了
o我们得到的例子

$$231 + 24 + 23 + 33 + 62 + 6 + 5 + 7 + 6 = 397.$$

计算上述总和对 $10^9 + 7$ 取模。

输入

输入的第一行包含一个整数 n ($1 \leq n \leq 10^6$), 表示字符串的长度。第二行包含长度为 n 的字符串，由数字 1-9 组成。

输出

输出一个整数，所有表达式之和以 $10^9 + 7$ 为模。

例子

对于输入数据：

3
231

正确的结果是：

397

F – Fix the Coloring

Page 1 of 2

Memory limit: 1024 MB

Time limit: 6 s

EUC 2026

8.02.2026

y 给定一个由 n 顶点和 m 边组成的无向图。每个顶点的颜色为黑色或白色。你
w 希望颜色合适。也就是说，由边连接的两个顶点不应该具有相同的颜色。某些顶点对可能不满足此属性。你
想通过重新绘制一些来解决这个问题

v 文章。但是，您只有红色油漆，因此您将一些顶点重新绘制为红色。

但即使手头有三种颜色，固定颜色也是不可能的。例如，当图表
c 包含一个大团（即一组顶点，其中所有对都通过边连接）。你决定允许
该图中没有一个非平凡的集团将其所有顶点都漆成红色。

因此，您的任务是检查是否可以通过以下方式将某些顶点绘制为红色：

- 连接红色顶点的所有边形成一个团。
- 所有剩余的边连接不同颜色的顶点。

输入

输入的第一行包含一个整数 t ($1 \leq t \leq 1000$) 表示测试用例的数量。

每个测试用例都以包含两个整数 n 和 m ($1 \leq n, m \leq 3000$) 的行开头，表示数字
of 图中的顶点和边。顶点编号从 1 到 n 。

第二行包含一串 n 字符 B, W；第 i 个字符表示第 i 个顶点的颜色
B 黑色，W 白色）。

接下来的 m 行包含图的边，每条边由一对顶点编号 a_i, b_i ($1 \leq$ 描述
 $a_i, b_i \leq n, a_i \neq b_i$)。每个无序对 u, v 最多会在文本案例中出现一次。

所有测试用例的值 n 和 m 的总和不超过 3000。

输出

输出所有 t 测试用例的答案。

如果无法固定颜色，则答案是由单词 NO 组成的一行。否则，答案应由两行组成：第一行应包含一个单词
YES，第二行应包含一个由 n 字符组成的字符串 B、W、R 表示顶点的新颜色（其中 R 表示红色）。如果有多个
有效解，则可以输出其中任意一个。

F – Fix the Coloring

Page 2 of 2

Memory limit: 1024 MB
Time limit: 6 s

EUC 2026
8.02.2026

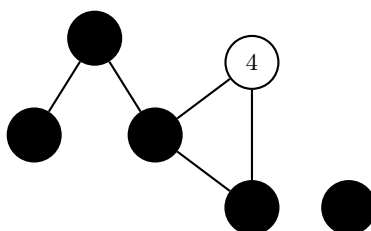
例子

对于输入数据:

```
2
6 7
BBBWBB
1 2
2 3
3 1
2 4
2 5
4 5
5 6
6 7
BBBBBB
1 2
2 3
3 1
2 4
2 5
4 5
5 6
```

可能的正确结果是:

```
YES
RRRWBR
NO
```



Explanation: 在第一个测试用例中，一个解决方案是重新绘制 clique 1、2、3 和顶点 6。另一种解决方案是重新绘制团 1、3 和顶点 5。

G——良好的排列

Page 1 of 1

内存限制：1024 MB 时间限制：3 秒

EUC 2026
8.02.2026

给定一个长度为 n 的数组 $a_1, \dots, a_n$, 由 $\{1, \dots, n\} \cup \{-1\}$ 中的整数组成。

如果 $\{p_1, \dots, p_k\} = \{1, \dots, k\}$ 且 $p_{i+1} \geq p_i - 1$, 则整数序列 $p_1, \dots, p_k$ 称为 *a good permutation* 所有 $1 \leq i < k$ 的旧值。

您要回答 q 询问。每个查询由一对整数 (l, r) 指定。对于这样的查询, 检查是否可以用正整数替换子数组 $a_l, \dots, a_r$ 中的所有 -1 , 使得该子数组 b 得到一个很好的排列。

输入

输入的第一行包含两个整数 n 和 q ($1 \leq n, q \leq 2 \cdot 10^5$), 表示数组的长度和查询数。

第二行包含一系列 n 整数 $a_1, a_2, \dots, a_n$ ($-1 \leq a_i \leq n, a_i \neq 0$) 表示元素 of 数组。

接下来的 q 行描述查询; 第 i 行包含两个整数 l_i 和 r_i ($1 \leq l_i \leq r_i \leq n$) 代表第 i 个查询。

输出

准确输出 q 行。第 i 行应包含第 i 个查询的答案, 即单词 YES 或 NO。

例子

For the input data:

```
5 3
-1 2 1 -1 5
1 5
2 5
2 4
```

the correct result is:

```
YES
NO
YES
```

Explanation: 在第一个查询中, 我们可以得到一个很好的排列 3, 2, 1, 4, 5。在第二个查询中, 没有好的排列 p_4 个元素的排列可以包含元素 5。在第三个查询中, 我们可以得到一个很好的排列 2, 1, 3。

而对于输入数据:

```
2 2
1 1
1 2
1 1
```

正确的结果是: NO
YES

H – House

Page 1 of 2

Memory limit: 1024 MB

Time limit: 2 s

EUC 2026

8.02.2026

Byteasar 想建造一座木屋。他有 n 块长度为 $a_1, \dots, a_n$ 且高度相等的木头。木屋的墙壁将使用帽子木材。

Byteasar 决定房子的地板是长方形，他想把它做成可能的。对于尺寸为 $x \times y$ 的矩形地板，房子的面积将为 $x \cdot y$ 。

房子有四堵墙，每堵墙都是矩形。对于每面墙，Byteasar 将使用精确的 k 木板，将被堆叠在彼此之上。因此，他需要 $2k$ 块长度为 x 的木板和 $2k$ 块长度为 y 的木板。

Byteasar 可以随心所欲地将木块切割成木板。因此，从一块木头 length a_i 他可以获得任何实际长度为 $b_1, \dots, b_m$ 的木板序列，只要 $b_1 + \dots + b_m \leq a_i$ 长。拜特萨不必使用所有木材；任何剩余的废料可能会保留。

帮助他计算他可以获得的房子的最大面积。

输入

输入的第一行包含两个整数 n 和 k ($1 \leq n \leq 1000$; $1 \leq k \leq 30$)，表示数量木板和墙壁的高度。

第二行有一个 n 整数序列 $a_1, \dots, a_n$ ($1 \leq a_i \leq 1000$)，表示木板。

输出

写一个实数，房子的最大可能面积（即 $x \cdot y$ 的最大值，前提是

Byteasar 可以获得 $2k$ 长度为 x 的木板和 $2k$ 长度为 y 的木板。

允许的相对或绝对误差为 10^{-9} 。也就是说，如果你输出 S ，正确的精确结果是 R ，那么它必须满足 $|S - R| \leq 10^{-9} \cdot \max(1, R)$ 。您最多可以打印小数点后 20 位数字。

H – House

Page 2 of 2

Memory limit: 1024 MB
Time limit: 2 s

EUC 2026
8.02.2026

例子

对于输入数据:

1 5
10

正确的结果是:
0.25

而对于输入数据:

5 1
6 7 1 3 2

正确的结果是:
12

对于输入数据:

5 7
1 4 5 7 5

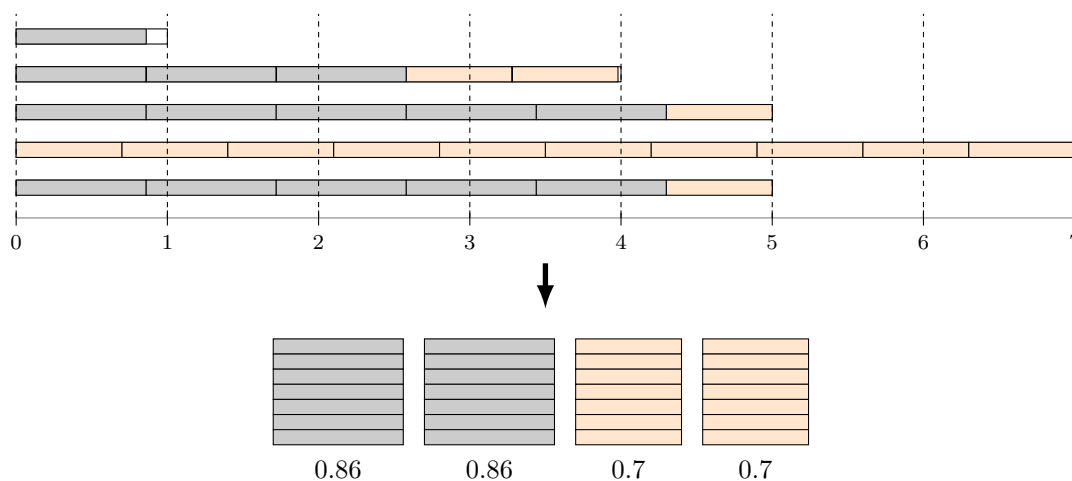
正确的结果是:
0.602

说明: 在第一个示例中, 我们需要 20 块木板 (每面墙 5 块)。最好的办法是将唯一的一块木头切成20等份, 每份长度为0.5, 并建造一座面积为 $0.5 \cdot 0.5 = 0.25$ 的房子。

在第二个示例中, 我们有两个不同的解决方案, 可以获取面积为 12 的房屋:

- 对于尺寸为 6×2 的房子, 我们需要将长度 7 的一块缩短为 6, 将长度 3 的一块缩短为 2。
- 对于尺寸为 4×3 的房屋, 我们需要将一块 7 切成长度为 3 和 4 的木板, 并将一块 6 缩短为 4。

在第三个示例中, 最佳房屋的尺寸为 0.7×0.86 。下图为如何获取 14 块长度为 0.86 的木板、14 块长度为 0.7 的木板以及两块长度为 0.14 和 0.02 的碎片:



I – Palindromes

Page 1 of 1

Memory limit: 1024 MB
Time limit: 3 s

EUC 2026
8.02.2026

在这个问题中，我们考虑二进制字母表 $\{a, b\}$ 上的字符串。如果一个字符串读作 *palindrome*，则该字符串被称为 *palindrome*。从左到右和从右到左他都是一样的。例如，字符串 *a*、*aba* 和 *babbab* 是回文，但 *abab* 不是。

字母表 $\{a, b\}$ 上存在长度为 N 的未知二进制字符串 s 。我们只知道它的前缀即长度为 $a_1, \dots, a_n$ 的初始片段是回文。有多少种方法可以恢复字符串 s ？

你的答案可能相当大；它足以返回模 m 的余数。

请注意，除了 length 的前缀之外，所查找的字符串 s 可能还有其他回文前缀 $a_1, \dots, a_n$ 。

输入

第一行包含三个整数 N 、 n 和 m ($1 \leq N \leq 10^{18}$ 、 $1 \leq n \leq 500\,000$ 、 $2 \leq m \leq 10^9$)，分别表示字符串的长度、字符串的回文前缀的数量以及用于计算的数量。

第二行包含严格递增的 n 整数 $a_1, a_2, \dots, a_n$ (序列，其中 $1 \leq a_i \leq N$) 表示所查找字符串的回文前缀的长度。

输出

输出应包含一个非负整数，即具有回文的长度为 N 的字符串的数量，对每个长度 $a_1, \dots, a_n$ 的修正，模 m 。

例子

对于输入数据：

10 3 100
2 5 9

正确的结果是：

8

解释：有 8 个长度为 10 的字符串，其长度为 2、5 和 9 的前缀是回文：

1. aaaaaaaaaa
2. aaaaaaaaaab
3. aabaaabaaa
4. aabaaabaab
5. bbabbbabba
6. bbabbbabbb
7. bbbbbbba
8. bbbbbbba

J – Jewel Guards

Page 1 of 1

Memory limit: 1024 MB

Time limit: 2 s

EUC 2026

8.02.2026

我们正在组织一个珠宝展览。您的任务是确保珠宝在夜间安全。

有 k 守卫可用。对于每个警卫，都知道该警卫何时可以在夜间工作。我们假设每个晚上被分为 n 个时间单位。如果在至少 m 时间内，珠宝将被视为 *safe* 夜里，他们至少有 *two* 个警卫看守。

为了避免守卫被贿赂，您需要每晚选择不同的守卫子集，以便珠宝每晚都很安全。计算存在多少个这样的子集。

输入

第一行包含三个整数 n 、 k 和 m ($1 \leq n \leq 10^9$ 、 $2 \leq k \leq 20$ 、 $1 \leq m \leq n$) 表示数字 n 为夜间的时间单位、警卫人数以及夜间的时间单位数。东边两个守卫必须看守珠宝。

接下来的 k 行描述了每个警卫可以工作的时间间隔。每行以一个非负整数 c_i ($i \in \{1, \dots, k\}$) 开头，表示时间间隔的数量。接下来是 c_i 对整数 $a_{i,j}$ 、 $b_{i,j}$ ($j \in \{1, \dots, c_i\}$)，它们描述间隔 $[a_{i,j}, b_{i,j}]$ ，使得第 i 个防护从时间单位 $a_{i,j}$ 到时间单位 $b_{i,j}$ (含) 工作。间隔是成对不相交的，并按左端点递增的顺序列出。

您可以假设 $c_1 + c_2 + \dots + c_k \leq 10^6$ 。

输出

输出应包含一个非负整数，即可以选择的警卫子集的数量，使得珠宝在夜间是安全的。

例子

对于输入数据：

正确的结果是：2

```
10 3 6
2 1 4 8 10
3 2 3 4 5 10 10
3 1 2 4 5 7 10
```

解释：寻找的守卫子集是 $\{1, 3\}$ 和 $\{1, 2, 3\}$ 。第一名和第三名守卫都在六个时间单位内观看珠宝， $\{1, 2, 4, 8, 9, 10\}$ 。如果选择了所有三个守卫，则在八个时间单位 $\{1, 2, 3, 4, 5, 8, 9, 10\}$ 内，至少有两名守卫监视珠宝。

K – 多集方差

Page 1 of 2

内存限制：1024 MB 时间限制：2 秒

EUC 2026
8.02.2026

我们有 n 个整数多重集。我们通过从整个输入中获取每个多重集来创建一个非空多重集——任意多次（可能 0 次）。

例如，根据输入多重集 $\{1, 3\}$, $\{1, 2, 2, 3\}$, $\{3, 5\}$ ，我们可以创建多重集 $M = \{1, 3, 1, 3, 1, 2, 2, 3\}$ 。第一个项目选两次，第二个项目选一次，第三个项目不选。

我们可以实现的多重集的最小和最大方差是多少？

回想一下，多重集 $X = \{x_1, x_2, \dots, x_k\}$ 的均值 $E(X)$ 和方差 $\text{Var}(X)$ 定义如下：

$$E(X) = \frac{1}{k} \sum_{i=1}^k x_i, \quad \text{Var}(X) = \frac{1}{k} \sum_{i=1}^k (x_i - E(X))^2.$$

例如，上述多重集 M 的均值和方差为：

$$E(M) = \frac{1}{8}(1 + 3 + 1 + 3 + 1 + 2 + 2 + 3) = \frac{16}{8} = 2,$$

$$\text{Var}(M) = \frac{1}{8}((-1)^2 + 1^2 + (-1)^2 + 1^2 + (-1)^2 + 0^2 + 0^2 + 1^2) = \frac{6}{8} = \frac{3}{4}.$$

输入

输入的第一行包含一个整数 t ($1 \leq t \leq 1000$)，表示测试用例的数量。

每个测试用例都以包含一个整数 n ($1 \leq n \leq 10^5$) 的行开始，表示 **multise** 的数量。接下来的 n 行描述多重集。每行以一个整数 k ($1 \leq k \leq 10$) 开头，表示 t 多重集，后跟 $[-2 \cdot 10^5, 2 \cdot 10^5]$ 范围内的 k 整数序列，表示 t 他是多组的。

所有测试用例的所有多重集的大小总和不超过 10^6 。

ts

输出

Output 精确包含后续测试答案的 t 行 t 例。

答案由两个用空格分隔的实数组成，即最小方差和最大方差。这一允许的相对或绝对误差为 10^{-11} 。也就是说，如果你输出 S 并且正确的精确结果是 R ，那么它必须保持 $|S - R| \leq 10^{-11} \cdot \max(1, R)$ 。您最多可以打印小数点后 20 位数字。

可以证明，总存在一个使方差最小的解和一个使方差最小的解——达到最大方差。

K – Multiset Variance

Page 2 of 2

Memory limit: 1024 MB
Time limit: 2 s

EUC 2026
8.02.2026

例子

对于输入数据:

```
3
3
2 1 3
4 1 2 2 3
2 3 5
2
3 -3 0 0
3 -2 -2 1
2
2 1 3
1 5
```

正确的结果是:

```
0.5 2
2 2
0 2.7777777777778
```

Explanation: 在第一个测试用例中, 多重集 $\{1, 2, 2, 3\}$ 实现了最小方差 (等于 $\frac{1}{2}$), 求多重集 $\{1, 3, 3, 5\}$ 的最大值 (等于 2)。

在第二个测试用例中, 获得多重集的最小和最大方差 (等于 2)

$\{-3, 0, 0, -2, -2, 1\}$ 。

在第三个测试用例中, 多重集 $\{5\}$ 实现了最小方差 (等于 0), 最大方差为 (等于 $\frac{25}{9}$) 对于多重集 $\{1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 5, 5, 5, 5, 5, 5, 5\}$ 。